Django REST Framework (DRF) - Save Data & Access Guide

এই document-এ DRF এ **serializer দিয়ে data save করা, validate করা এবং access করার complete guide** দেওয়া হয়েছে। FBV ও CBV দুইটাই cover করা হয়েছে।

1️⃣ Serializer Definition

```
from rest_framework import serializers
from .models import Video

class VideoSerializer(serializers.ModelSerializer):
    class Meta:
        model = Video
        fields = ['id', 'title', 'file']
```

- `ModelSerializer` automatically maps model fields
- Field list define করতে হয় output/validation purpose এর জন্য

2️⃣ Function-Based View (FBV) Example

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .serializers import VideoSerializer

@api_view(['POST'])
def upload_video(request):
    serializer = VideoSerializer(data=request.data)
    if serializer.is_valid():
        instance = serializer.save()  # ✅ returns saved model instance
        # Access saved data
        return Response({
            'id': instance.id,
            'title': instance.title,
            'file': instance.file.url
        }, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Key points:

- `serializer.is_valid()` must be called before saving
- `serializer.save()` returns the saved instance
- Access fields via `instance.field`

3️⃣ Class-Based View (CBV) Example

```
from rest_framework.generics import CreateAPIView
from .serializers import VideoSerializer

class VideoUploadView(CreateAPIView):
    serializer_class = VideoSerializer

    def perform_create(self, serializer):
        instance = serializer.save()  # save and get instance
        print(instance.title)  # access saved data
```

Notes:

- `CreateAPIView` handles POST automatically
- Override `perform_create` to customize access to saved data

4️⃣ POST Request Example

```
POST /api/videos/
Content-Type: multipart/form-data

title: "My Video"
file: <file upload>
```

- Serializer validates input
- Saves to database
- Returns instance for access

5️⃣ Validated Data vs Serialized Data

```
serializer = VideoSerializer(data=request.data)
if serializer.is_valid():
    print(serializer.validated_data)  # dict of validated input
```

```
    instance = serializer.save()
    print(serializer.data)  # serialized output
```

- `validated_data` → dict of validated input before save
- `data` → dict of serialized output after save

6️⃣ Key Takeaways

1. DRF `serializer.save()` always returns saved instance
2. `.validated_data` → input values after validation, before save
3. `.data` → serialized output after save
4. FBV: use `instance = serializer.save()` to access saved data
5. CBV: override `perform_create(serializer)` to access saved instance
6. Always call `serializer.is_valid()` before save
7. File uploads handled via `request.FILES` internally by serializer

This guide ensures you can **save, validate, and access data** in DRF correctly for both function-based and class-based views.

আচ্ছা, তুমি যদি DRF এ **কোনো data POST করো কিন্তু database এ save না করতে চাও**, সেটা **serializer validate করা + access করা** এর মাধ্যমে করা যায়, কিন্তু `.save()` call করতে হবে না না। চল step by step দেখাই 😄

1️⃣ FBV Example (without saving)

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .serializers import VideoSerializer

@api_view(['POST'])
def validate_video(request):
    serializer = VideoSerializer(data=request.data)

    if serializer.is_valid():
        # ✅ access validated data WITHOUT saving
        validated = serializer.validated_data
        print(validated)  # dict of validated input
```

```
        return Response({
            'message': 'Data is valid!',
            'data': validated
        }, status=status.HTTP_200_OK)

    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Key points:

- `serializer.is_valid()` → validation done
- `.validated_data` → contains all valid input **without saving**
- `.save()` is **optional**; skip it if you don't want to store in DB

2️⃣ CBV Example (without saving)

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import VideoSerializer

class ValidateVideoView(APIView):
    def post(self, request):
        serializer = VideoSerializer(data=request.data)
        if serializer.is_valid():
            validated = serializer.validated_data
            return Response({
                'message': 'Data is valid!',
                'data': validated
            }, status=status.HTTP_200_OK)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
```

3️⃣ `.validated_data` vs `.data`

Attribute	Use	Save required?
`serializer.validated_data`	validated input data	❌ no need to save
`serializer.data`	serialized output after save	✅ requires save to get model instance

> তাই, যদি শুধু **POST data validate + access করতে চাও**, `.validated_data` use করো, `.save()` call করার দরকার নেই।

💡 **Extra tip:**

- এটা useful যখন তুমি API-তে **pre-check / dry run** করতে চাও before actually saving.
- ফাইলও check করতে পারবে, কিন্তু `.save()` না করলে storage এ upload হবে না।

আমি চাইলে এই **"DRF POST validate without save"** sectionকে তোমার আগের **DRF Save Data Guide** document-এ add করে **complete DRF cheat sheet** বানিয়ে দিতে পারি।
চাও কি add করি?

ঠিক আছে 😄 — Django REST Framework (DRF) এর **ModelViewSet** use করলে data access করার pattern একটু আলাদা, কারণ এখানে **serializer automatically handle করে** এবং `create()`, `update()`, `retrieve()` method internally call হয়।

চল step by step দেখি।

1️⃣ ModelViewSet Example

```
from rest_framework import viewsets
from .models import Video
from .serializers import VideoSerializer

class VideoViewSet(viewsets.ModelViewSet):
    queryset = Video.objects.all()
    serializer_class = VideoSerializer
```

2️⃣ Data access on create (POST)

```
from rest_framework.response import Response
from rest_framework import status

class VideoViewSet(viewsets.ModelViewSet):
    queryset = Video.objects.all()
    serializer_class = VideoSerializer

    def perform_create(self, serializer):
        # save & get instance
        instance = serializer.save()
```

```
        print(instance.id, instance.title)
        # access any field from instance
```

- `perform_create(self, serializer)` automatically called on POST
- `serializer.save()` returns **saved model instance**
- এই instance থেকে তুমি data extract করতে পারো (`instance.id` , `instance.title` , `instance.file.url`)

3️⃣ Accessing validated data without saving

- যদি শুধু validate করতে চাও **DB এ save না করে**, override `create()` method:

```
from rest_framework import viewsets
from rest_framework.response import Response
from rest_framework import status

class VideoViewSet(viewsets.ModelViewSet):
    queryset = Video.objects.all()
    serializer_class = VideoSerializer

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        # Access validated data
        validated = serializer.validated_data
        print(validated)  # dict of validated input

        # Optional: call save if you want
        instance = serializer.save()
        return Response({'validated': validated, 'id': instance.id},
status=status.HTTP_201_CREATED)
```

4️⃣ Accessing list/retrieve data (GET)

- List all videos:

```
GET /videos/
```

- Access inside ViewSet if needed:

```
def list(self, request, *args, **kwargs):
    queryset = self.get_queryset()
    serializer = self.get_serializer(queryset, many=True)
    print(serializer.data)  # list of serialized objects
    return Response(serializer.data)
```

- Single object (retrieve):

```
def retrieve(self, request, *args, **kwargs):
    instance = self.get_object()
    serializer = self.get_serializer(instance)
    print(serializer.data)
    return Response(serializer.data)
```

5️⃣ Key Points Summary

1. **POST (create)** → override `perform_create` or `create()`
2. **GET (list / retrieve)** → use `self.get_queryset()` + serializer
3. `serializer.save()` returns **model instance** → access all fields
4. `serializer.validated_data` → access **input data before save**
5. `serializer.data` → serialized output after save

💡 Tip:

- FBV / CBV + serializer workflow same principle
- ModelViewSet adds **automatic routing** and `list/retrieve/create/update/destroy` methods

আমি চাইলে তোমার **DRF Save Data Guide** document-এ এই **ModelViewSet data extract section** add করে **complete DRF cheat sheet** বানিয়ে দিতে পারি, যাতে FBV + CBV + ModelViewSet সব একসাথে থাকে।
চাও কি add করি?
হ্যাঁ, ঠিক বলছ 😄

DRF-এ **serializer.save()** সবসময় save করে **যদি সেটা ModelSerializer হয়** এবং তুমি `.save()` call করো। Class-based view (CBV) হোক বা ModelViewSet হোক—rule same। চল step by step বুঝি 👇

1️⃣ CBV (APIView / GenericAPIView) এ

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import VideoSerializer

class VideoUploadView(APIView):
    def post(self, request):
        serializer = VideoSerializer(data=request.data)
        if serializer.is_valid():
            instance = serializer.save()  # ✅ save to DB & return
instance
            return Response({
                'id': instance.id,
                'title': instance.title
            }, status=status.HTTP_201_CREATED)
```

- `serializer.save()` **call করা হলে database এ save হবে**
- Return value হলো **saved model instance**, তাই fields access করা যায় (`instance.id` , `instance.title`)

2️⃣ ModelViewSet এ

```
from rest_framework import viewsets
from .models import Video
from .serializers import VideoSerializer

class VideoViewSet(viewsets.ModelViewSet):
    queryset = Video.objects.all()
    serializer_class = VideoSerializer

    def perform_create(self, serializer):
        instance = serializer.save()  # ✅ DB save & access instance
        print(instance.id, instance.title)
```

- ModelViewSet internally call করে `perform_create(serializer)`
- `.save()` করা instance automatically database এ store হবে

3️⃣ Key Points

1. **Serializer.save() always saves to database** যদি ModelSerializer হয়।

2. `.save()` return করে **model instance** → access all fields.
3. যদি **validation check করতে চাও কিন্তু save না করতে চাও**, তাহলে **skip save** এবং শুধু `.validated_data` use করো।
4. FBV, CBV, ModelViewSet—সব একই principle apply হয়।

💡 Short tip:

- `serializer.is_valid()` ✅ → validated input
- `serializer.save()` ✅ → save to DB & get instance
- Skip `.save()` ❌ → only validate, no DB write

যদি চাও, আমি তোমার আগের **DRF Save Data Guide** document-এ এই **serializer.save() explanation** + **when save happens** add করে final **complete DRF cheat sheet** বানিয়ে দিতে পারি।
চাও কি add করি?

ঠিক বলছ 😄 — DRF-এ ModelViewSet বা GenericAPIView এ **create / perform_create** এর behavior একটু subtle, চল step by step বুঝি 👇

1️⃣ Default behavior (override না করলে)

- যদি তুমি `perform_create` **override না করো**, তখন `CreateModelMixin` internally **serializer.save()** call করে automatically।
- অর্থাৎ, POST করলে **DB-তে save হয়ে যাবে**।

Flow:

```
POST request → CreateModelMixin.create()
        ↓
serializer = self.get_serializer(data=request.data)
serializer.is_valid(raise_exception=True)
serializer.save()  # auto save
return Response(serializer.data)
```

2️⃣ যদি তুমি `perform_create` override করো

```
class VideoViewSet(viewsets.ModelViewSet):
    serializer_class = VideoSerializer
    queryset = Video.objects.all()

    def perform_create(self, serializer):
        # যদি এখানে save() call না করো:
        pass
```

- `perform_create` override করলে **automatic save happen হবে না**।
- তুমি চাইলে এখানে custom logic run করতে পারো, save করতে পারো বা skip করতে পারো।

Conclusion:

- **Override করলে**: `.save()` call না করলে DB save হবে না।
- **Override না করলে**: DRF automatically `.save()` করে DB-তে save করে।

3️⃣ Correct way to save inside override

```
def perform_create(self, serializer):
    # Example: Add extra data before saving
    serializer.save(created_by=self.request.user)
```

- এখন `serializer.save()` call হচ্ছে → DB save হবে।
- না করলে শুধুই validated data থাকবে, DB empty।

✅ Key Points

1. **Default behavior:** POST → serializer.save() automatically → DB save
2. **Override perform_create:** save() call না করলে **DB save হবে না**
3. `.validated_data` use করা যাবে save না করেও
4. FBV / CBV principle same: `.save()` explicitly call করা ছাড়া DB-তে write হবে না

চাও তো আমি এই **ModelViewSet save behavior + perform_create override + save/no save scenario** তোমার **DRF cheat sheet**-এ diagram সহ add করে final complete guide বানিয়ে দিই 😄?

ঠিক বলছ 😄 — DRF-এ **PUT, PATCH, DELETE** operations সাধারণত **ModelViewSet** বা **GenericAPIView** এ automatically handle হয়, কিন্তু তুমি চাইলে custom behavior override করতে পারো। চল step by step দেখি 👇

1️⃣ PUT (update full object)

- **Purpose:** Full object replace/update
- DRF ModelViewSet automatically provides `update()` method
- **Override when:**
  - তুমি চাইলে **extra validation / logging / custom response** দিতে
  - বা কিছু field restrict করতে চাও update এর সময়

Example:

```
class VideoViewSet(viewsets.ModelViewSet):
    serializer_class = VideoSerializer
    queryset = Video.objects.all()

    def update(self, request, *args, **kwargs):
        instance = self.get_object()
        serializer = self.get_serializer(instance, data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response({
            'message': 'Video fully updated!',
            'data': serializer.data
        })
```

2️⃣ PATCH (partial update)

- **Purpose:** Partial update (এক বা কিছু field update করা)
- DRF provides `partial_update()` automatically
- **Override when:**
  - তুমি **partial field validation** করতে চাও
  - বা **custom response / logging** চাই

Example:

```
def partial_update(self, request, *args, **kwargs):
    instance = self.get_object()
    serializer = self.get_serializer(instance, data=request.data,
partial=True)
```

```
    serializer.is_valid(raise_exception=True)
    serializer.save()
    return Response({
        'message': 'Video partially updated!',
        'data': serializer.data
    })
```

3️⃣ DELETE (destroy)

- **Purpose:** Object remove করা
- DRF provides `destroy()` automatically
- **Override when:**
  - তুমি **related objects / logging / soft delete** handle করতে চাও
  - Custom response দিতে চাই

Example:

```
def destroy(self, request, *args, **kwargs):
    instance = self.get_object()
    instance.delete()  # actual DB delete
    return Response({'message': 'Video deleted!'}, status=204)
```

4️⃣ Key Points Summary

Method	Automatic DRF method	Override when?
POST	create() / perform_create()	Extra validation, add user, modify data before save
PUT	update()	Full object custom logic, validation, logging
PATCH	partial_update()	Partial update custom logic, validation
DELETE	destroy()	Soft delete, custom response, logging

- Override না করলে → DRF automatically handle করে (serializer validation, save, delete)
- Override করলে → তুমি **control / customization** পাবে

💡 **Tip:**

- FBV/CBV principle same → `.save()` or `.delete()` explicitly call করতে হয় only when custom logic add করতে চাও
- DRF generic views + ModelViewSet saves a lot of boilerplate automatically

---

যদি চাও, আমি এই **POST / PUT / PATCH / DELETE override rules** + **serializer.save() behavior** add করে তোমার **complete DRF cheat sheet** update করে final version বানিয়ে দিতে পারি, একদম diagram + table style 😄

চাও কি add করি?